

Exercise 01:

Two words are anagrams if one word can be formed by rearranging all the letters of the other word, for example: treason, senator. A word is represented as a linked list with one letter per node of the list. Write a method that, given two linked lists w1 and w2, each representing a word of lowercase letters, returns true if the words are anagrams and false if they are not.

Base your algorithm on the following: for each letter in w1, search w2 for it; if found, delete it and continue; otherwise, return false.

```
import java.util.LinkedList;
public class Exercise22 {
    public static boolean isAnagram(LinkedList<Character> w1,
                                    LinkedList<Character> w2) {
        if (w1.size() != w2.size())
            return false;
        LinkedList<Character> temp = new LinkedList<>(w2);
        for (char c : w1) {
            if (temp.contains(c)) {
                temp.remove((Character)c);
            }
            else {
                return false;
            }
        }
        return true;
    }
    public static void main(String[] args) {
        LinkedList<Character> word1 = new LinkedList<>();
        word1.add('t');
        word1.add('r');
        word1.add('e');
        word1.add('a');
        word1.add('s');
        word1.add('o');
        word1.add('n');
        LinkedList<Character> word2 = new LinkedList<>();
        word2.add('s');
        word2.add('e');
        word2.add('n');
        word2.add('a');
        word2.add('t');
        word2.add('o');
        word2.add('r');
        System.out.println(isAnagram(word1, word2));
    }
}
```

Output

true

Exercise 02:

Add a method in the class LinkedList named sumValues() that returns the sum of the values in all nodes of a linked list. Create an application in which you:

1. Create a linked list and add to it the values 6, 4, 10, 9, 1.
2. Call the method sumValues to output the sum of the values in the linked list (the sum will be 30).

```
import java.util.LinkedList;
public class MyLinkedList {
    LinkedList<Integer> list = new LinkedList<>();
    public int sumValues() {
        int sum = 0;
        for (int value : list) {
            sum += value;
        }
        return sum;
    }
    public static void main(String[] args) {
        MyLinkedList obj = new MyLinkedList();
        obj.list.add(6);
        obj.list.add(4);
        obj.list.add(10);
        obj.list.add(9);
        obj.list.add(1);
        System.out.println("Sum = " + obj.sumValues());
    }
}
```

Output

Sum = 30

Exercise 03:

Add a method in the application named sumNumbers() that receives as a parameter from the main method a linked list and returns the sum of the values in all nodes.

Create an application in which you:

1. Create a linked list and add to it the values 6, 4, 10, 9, 1.
2. Call the method sumNumbers to output the sum of the values in the linked list (the sum will be 30).

```
import java.util.LinkedList;
public class Exercise24 {
    public static int sumNumbers(LinkedList<Integer> list) {
        int sum = 0;
        for (int number : list) {
            sum += number;
        }
        return sum;
    }
    public static void main(String[] args) {
        LinkedList<Integer> numbers = new LinkedList<>();
        numbers.add(6);
        numbers.add(4);
        numbers.add(10);
        numbers.add(9);
        numbers.add(1);
        System.out.println("Sum = " + sumNumbers(numbers));
    }
}
```

Output
Sum = 30

Exercise 04:

A frequently occurring problem that is detected by compilers is to determine if the brackets of an expression are properly balanced and included one in the other. For example, the expression $((a+b)*c)+(u*v)$ is well balanced and properly written while the expressions $(a+b)*c$ and $(a+b)*c)+b($ are not. Write an application in which you:

- Write a boolean method named `check` that receives as a parameter a `String` expression and checks if the expression is well balanced and properly written or not. The header of the method is: **public static boolean check(String expression)**
- Write a main method to test the above method.

Note: Use the stack ADT operations to solve this problem. You are not allowed to use arrays or counters.

```
import java.util.Stack;
import java.util.Scanner;
public class Exercise25 {
    public static boolean check(String expression) {
        Stack<Character> stack = new Stack<>();
        for (int i = 0; i < expression.length(); i++) {
            char ch = expression.charAt(i);
            // Add opening brackets
            if (ch == '(') {
                stack.push(ch);
            }
            // Check closing bracket
            else if (ch == ')') {
                if (stack.empty()) {
                    return false;
                }
                stack.pop();
            }
        }
        return stack.empty();
    }
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter expression: ");
        String exp = input.nextLine();
        if (check(exp))
            System.out.println("Balanced expression");
        else
            System.out.println("Not balanced expression");
    }
}
```

Output 1
Enter expression: $((a+b)*c)+(u*v)$
Balanced expression

Output 2
Enter expression: $(a+b)*c$
Not balanced expression

Exercise 05:

In a bank, customers wait in a queue (q0) for their turn to complete a transaction. Customers who come first are served first. At a certain point when there are lots of customers waiting, a decision is taken to split the queue elements into two empty queues q1 and q2. The customers are split based on the alphabetical order of their names. Customers with first name starting A-H are added to q1 while the others (first name starting I- Z) are added to q2.

In an application, write a method named splitQueue (in an application) that accepts as parameters 3 queues, the original queue (q0) and two empty queues (q1 and q2) to be filled as described above. Write also a main method in which you read the names of customers from the user, fill them into q0 and call the method splitQueue to split q0 elements into q1 and q2. After the split display the elements of q1 first then those of q2. The header of the method is:

public static void splitQueue (Queue q0, Queue q1, Queue q2)

Note: Make sure you do the necessary modifications to allow the queue to store Strings (first names).

```
import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;
public class Exercise26 {
    public static void splitQueue(Queue<String> q0, Queue<String> q1, Queue<String> q2) {
        while (!q0.isEmpty()) {
            String name = q0.remove();
            char firstLetter = name.charAt(0);
            if (firstLetter >= 'A' && firstLetter <= 'H') {
                q1.add(name);
            }
            else {
                q2.add(name);
            }
        }
    }
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Queue<String> q0 = new LinkedList<>();
        Queue<String> q1 = new LinkedList<>();
        Queue<String> q2 = new LinkedList<>();
        System.out.print("Enter number of customers: ");
        int n = input.nextInt();
        input.nextLine();
        for (int i = 0; i < n; i++) {
            System.out.print("Enter customer name: ");
            String name = input.nextLine();
            q0.add(name);
        }
        splitQueue(q0, q1, q2);
        System.out.println("\nQueue 1 (A-H):");
        while (!q1.isEmpty()) {
            System.out.println(q1.remove());
        }
        System.out.println("\nQueue 2 (I-Z):");
        while (!q2.isEmpty()) {
            System.out.println(q2.remove());
        }
    }
}
```

```
}
```

Output

Enter number of customers: 6

Enter customer name: Ali

Enter customer name: Sara

Enter customer name: Hassan

Enter customer name: Omar

Enter customer name: Ahmad

Enter customer name: Lina

Queue 1 (A-H):

Ali

Hassan

Ahmad

Exercise 06: (Big O)

Prove that $n^3 + 1000 = O(n^3)$

According to the definition of Big-O:

$f(n) \leq c \times g(n)$ where $f(n) = n^3 + 1000$ $g(n) = n^3$

We need to find constants c and n_0 .

Choose:

$c = 2$ and: $n_0 = 10$

For every $n \geq 10$ we have:

$n^3 \geq 1000$

Therefore $n^3 + 1000 \leq n^3 + n^3$

So $n^3 + 1000 \leq 2n^3$

This means $f(n) \leq c \times g(n)$

Therefore $n^3 + 1000 = O(n^3)$

Final Answer:

$c = 2$

$n_0 = 10$

Exercise 07:

Determine the running time of the following code in terms of Big-Oh notation. Show the steps in details.

```
public static void method(int n) {  
    int[] a = new int[n];  
    for (int i = 0; i < n; i++)  
        a[i] = in.nextInt();  
    for (int i = 0; i < n; i++)  
        System.out.println(a[i]);  
}
```

Step 1: Creating the array `int[] a = new int[n];` $O(1)$

Step 2: First loop `for (int i = 0; i < n; i++)` $\rightarrow O(n)$

Step 3: Second loop `for (int i = 0; i < n; i++)` $\rightarrow O(n)$

The operation inside the loop takes constant time:

Step 4: Total Running Time $O(1) + O(n) + O(n) \rightarrow O(n^2)$

EXERCISE 08: Recursive oddSum Method

Write a recursive method named oddSum that takes a positive odd integer n and returns the sum of odd integers from 1 to n.

Example , oddSum(5) = 1 + 3 + 5 = 9

```
import java.util.Scanner;
public class Exercise29 {
    public static int oddSum(int n) {
        if (n == 1)
            return 1;
        return n + oddSum(n - 2);
    }
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter an odd number: ");
        int n = input.nextInt();
        System.out.println("Sum = " + oddSum(n));
    }
}
```

Output:

Enter an odd number: 7

Sum = 16

Exercise 09:

Write a recursive method named oddSum that takes a positive odd integer n and returns the sum of odd integers from 1 to n. Start with a base case, and use temporary variables to debug your solution. You might find it helpful to print the value of n each time oddSum is invoked. Write a main method to test the above method.

Write a recursive method named oddSum that takes a positive odd integer n and returns the sum of odd integers from 1 to n.

Example: oddSum(7) = 1 + 3 + 5 + 7 = 16

```
import java.util.Scanner;
public class Exercise29 {
    public static int oddSum(int n) {
        if (n == 1)
            return 1;
        return n + oddSum(n - 2);
    }
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a positive odd number: ");
        int n = input.nextInt();
        System.out.println("sum of odd numbers = " + oddSum(n));
    }
}
```

Output:

Enter a positive odd number: 9

Sum of odd numbers = 25

Exercise 10:

A palindrome is a word that reads the same both forward and backward, like “otto” and “palindromeemordnilap”. Here’s one way to test whether a string is a palindrome:

A single letter is a palindrome, a two-letter word is a palindrome if the letters are the same, and any other word is a palindrome if the first letter is the same as the last and the middle is a palindrome.

Write a recursive method named isPalindrome that takes a string as a parameter and returns true

if the word is a palindrome and false otherwise.

Write a main method to test the above method.

```
import java.util.Scanner;
public class Exercise30 {
    public static boolean isPalindrome(String word) {
        if (word.length() <= 1)
            return true;
        if (word.charAt(0) != word.charAt(word.length() - 1))
            return false;
        return isPalindrome(word.substring(1, word.length() - 1));
    }
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a word: ");
        String word = input.nextLine();
        if (isPalindrome(word))
            System.out.println("The word is a palindrome");
        else
            System.out.println("The word is not a palindrome");
    }
}
```

Output 1:

Enter a word: otto

The word is a palindrome

Output 2:

Enter a word: hello

The word is not a palindrome

Exercise 11: (Tree&recursion)

Write a recursive method that takes a binary tree and checks if it is a full binary tree. A full binary tree is a tree in

which every node other than the leaves has exactly two children. In other words, a binary tree is full if each node

has 2 children or 0 children (leaf).

```
Node Class:
class TreeNode {
    int value;
    TreeNode left;
    TreeNode right;
    public TreeNode(int value) {
        this.value = value;
        left = null;
        right = null;
    }
}
```

Recursive Method:

```
public static boolean isFullBinaryTree(TreeNode root) {
    if (root == null)
        return true;
    if (root.left == null && root.right == null)
        return true;
    if (root.left == null || root.right == null)
        return false;
    return isFullBinaryTree(root.left) && isFullBinaryTree(root.right);
}
```

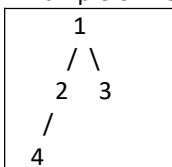
Test Program:

```
public class Exercise31 {
    public static void main(String[] args) {
        TreeNode root = new TreeNode(1);
        root.left = new TreeNode(2);
        root.right = new TreeNode(3);
        root.left.left = new TreeNode(4);
        root.left.right = new TreeNode(5);
        root.right.left = new TreeNode(6);
        root.right.right = new TreeNode(7);
        if (isFullBinaryTree(root))
            System.out.println("The tree is a full binary tree");
        else
            System.out.println("The tree is not a full binary tree");
    }
    public static boolean isFullBinaryTree(TreeNode root) {
        if (root == null)
            return true;
        if (root.left == null && root.right == null)
            return true;
        if (root.left == null || root.right == null)
            return false;
        return isFullBinaryTree(root.left) &&
            isFullBinaryTree(root.right);
    }
}
```

Output:

The tree is a full binary tree

Example of NOT Full Binary Tree:



Output:

The tree is not a full binary tree

EXERCISE 12: Graphs (Graph Traversal)

Create a graph using adjacency lists and implement Depth First Search (DFS) to visit all vertices.

```
import java.util.*;
public class Exercise32 {
    private int vertices;
    private LinkedList<Integer>[] adjacencyList;
    public Exercise32(int vertices) {
        this.vertices = vertices;
        adjacencyList = new LinkedList[vertices];
        for (int i = 0; i < vertices; i++)
            adjacencyList[i] = new LinkedList<>();
    }
    public void addEdge(int source, int destination) {
        adjacencyList[source].add(destination);
    }
    public void DFS(int start) {
        boolean[] visited = new boolean[vertices];
        DFSUtil(start, visited);
    }
    private void DFSUtil(int vertex, boolean[] visited) {
        visited[vertex] = true;
        System.out.print(vertex + " ");
        for (int neighbor : adjacencyList[vertex]) {
            if (!visited[neighbor])
                DFSUtil(neighbor, visited);
        }
    }
    public static void main(String[] args) {
        Exercise32 graph = new Exercise32(5);
        graph.addEdge(0,1);
        graph.addEdge(0,2);
        graph.addEdge(1,3);
        graph.addEdge(2,4);
        System.out.println("DFS Traversal:");
        graph.DFS(0);
    }
}
```

Output:

DFS Traversal: 0 1 3 2 4

EXERCISE 13: HashMap and HashTable

Create a program that stores student names and their grades using HashMap, then searches for a student grade.

```
import java.util.HashMap; import java.util.Scanner;
public class Exercise33 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        HashMap<String, Integer> students = new HashMap<>();
        students.put("Ali", 90);
        students.put("Ahmad", 85);
        students.put("Sara", 95);
        System.out.println("Students Grades:");
        System.out.println(students);
        System.out.print("Enter student name: ");
    }
}
```

```

String name = input.nextLine();
if (students.containsKey(name))
    System.out.println("Grade = " + students.get(name));
else
    System.out.println("Student not found");
}
}

```

Output:

```

Students Grades:
{Ali=90, Ahmad=85, Sara=95}
Enter student name:
Sara
Grade = 95

```

EXERCISE 14: Vector

Create a Vector that stores numbers. Add elements, remove an element, and display the vector.

```

import java.util.Vector;
public class Exercise34 {
    public static void main(String[] args) {
        Vector<Integer> numbers = new Vector<>();
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(40);
        System.out.println("Vector: " + numbers);
        numbers.remove(2);
        System.out.println("After removing element: " + numbers);
        System.out.println("First element: " + numbers.firstElement());
    }
}

```

Output

```

Vector: [10, 20, 30, 40]

```